

# Practice Problems on Persistence

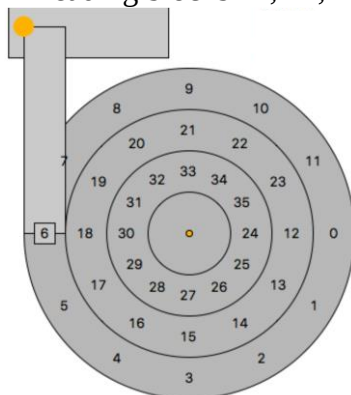
1. Provide one reason why a DMA-enabled device driver usually gives better performance over a non-DMA interrupt-driven device driver.
2. Which of the following statements is/are true regarding memory-mapped I/O?
  - A. The CPU accesses the device memory much like it accesses main memory.
  - B. The CPU uses separate architecture-specific instructions to access memory in the device.
  - C. Memory-mapped I/O cannot be used with a polling-based device driver.
  - D. Memory-mapped I/O can be used only with an interrupt-driven device driver.
3. Consider a file D1/F1 that is hard linked from another parent directory D2. Then the directory entry of this file (including the filename and inode number) in directory D1 must be exactly identical to the directory entry in directory D2. [T/F]
4. It is possible for a system that uses a disk buffer cache with FIFO as the buffer replacement policy to suffer from the Belady's anomaly. [T/F]
5. Reading files via memory mapping them avoids an extra copy of file data from kernel space buffers to user space buffers. [T/F]
6. A soft link can create a link between files across different file systems, whereas a hard link can only create links between a directory and a file within the same file system. [T/F]
7. Consider the process of opening a new file that does not exist (obviously, creating it during opening), via the "open" system call. Describe changes to all the in-memory and disk-based file system structures (e.g., file tables, inodes, and directories) that occur as part of this system call implementation. Write clearly, listing the structure that is changed, and the change made to it.
8. Now, suppose the process that has opened the file in the previous question proceeds to write 100 bytes into the file. Assume block size on disk is 512 bytes. Assume the OS uses a write-through disk buffer cache. List all the operations/changes to various datastructures that take place when the write operation successfully completes.
9. Repeat the above question for the implementation of the "link" system call, when linking to an existing file (not open from any process) in a directory from another new parent directory.
10. Repeat the above question for the implementation of the "dup" system call on a file descriptor.
11. Explain any 3 distinct advantages in the design of the Unix file system API where the file needs to be first opened and then accessed (for subsequent read/write operations) using the file descriptor (instead of accessing it using file path or file's inode number)
12. Consider a file system with 512-byte blocks. Assume an inode of a file holds pointers to N direct data blocks, and a pointer to a single indirect block. Further, assume that the single indirect block can hold pointers to M other data blocks. What is the maximum file size that can be supported by such an inode design?

13. Consider a primitive disk that has 50 tracks. A seek takes 5 ms (on average) to move the arm head per track. On an average, without optimization (to try to put sequential file data in blocks with close proximity) two logically consecutive blocks in a file will be about 10 tracks apart. however, with an optimization attempt to cluster related blocks, the mean inter-block distance can be reduced to 2 tracks. If the rotational latency is 50 ms and the transfer time is 15 ms per block, how long does it take to read a 50-block file in both cases
14. What are the problems with shortest seek time first (SSTF) method of disk scheduling and how they are addressed in an alternative strategy. Does any of the Disk scheduling strategies discussed in the course, guarantees to be always optimal (Justify your answer)?
15. Consider a FAT file system where disk is divided into M byte blocks, and every FAT entry can store an N bit block number. What is the maximum size of a disk partition that can be managed by such a FAT design?
16. What is the maximum file size supported by a file system with 16 direct blocks and a single, double, and triple indirection? The block size is 512 bytes. Disk block pointers/numbers can be stored in 4 bytes. You can leave the answer in exponential terms.
17. Consider a secondary storage system of size 2 TB, with 512-byte sized blocks. Assume that the filesystem uses a multilevel inode datastructure to track data blocks of a file. The inode has 64 bytes of space available to store pointers to data blocks, including a single indirect block, a double indirect block, and several direct blocks. What is the maximum file size that can be stored in such a file system?
14. Consider a filesystem managing a disk with block size  $2^b$  bytes, and disk block addresses of  $2^a$  bytes. The inode of a file contains n direct blocks, one single indirect block, one double indirect block, and one triple indirect block. What is the maximum size of a file (in bytes) that can be stored in this filesystem? Assume that the indirect blocks only store a sequence of disk addresses, and no other metadata.
15. The `fork` system call creates new entries in the open file table for the newly created child process. [T/F]
16. When a process opens a file that is already being read by another process, the file descriptors in both processes will point to the same open file table entry. [T/F]
17. Memory mapping a file using the `mmap` system call adds one or more entries to the page table of the process. [T/F]
18. The read system call to fetch data from a file always blocks the invoking process. [T/F]
19. During filesystem operations, if the filesystem implementation ensures that changes to data blocks of a file are flushed to disk before changes to metadata blocks (like inodes and bitmaps), then the filesystem will never be in an inconsistent state after a crash, and a filesystem checker need not be run to detect and fix any inconsistencies. [T/F]
20. Interrupt-based device drivers give superior performance to polling-based drivers because they eliminate the time spent by the CPU in copying data to and from the device hardware. [T/F]

21. When a process writes a block to the disk via a disk buffer cache using the write-back policy, the process invoking the write will block until the write is committed to disk. [T/F]
22. Study the following program segment and a) explain the output of the program (assume inclusion of necessary header files so that the code compiles and executes successfully) and mention a use-case of this type of code segment b) Draw figures that shows the status of file descriptor array of the processes involved, the global open file table, in-memory inode and the pointers linking these data structures just after the execution of fork() statement. Mention the content of any relevant information of these data structures in the figures. c) What changes should be reflected in the figures after execution of the close(in) statement by the processes involved.

```
int main(int argc, char *argv[]) {
    int in, pid, stat;
    char *args[] = {"grep", "xyz", NULL};
    in = open("file", O_RDONLY);
    pid=fork();
    if(pid==0){
        dup2(in, 0);
        close(in);
        execvp("grep", args);
    }
    close(in);
    waitpid(pid,&stat,0);
    printf(" The end\n");
    return 0;
}
```

23. What is the average service time to retrieve a single disk block (32 KByte size) from a random location of a disk that rotates at 10,000 RPM, have a data transfer rate of 10 MBytes/s and have an 8 ms average seek time? what is the bandwidth (bytes/sec) that can be achieved, if the application requests are randomly distributed?
24. What is the average service time to retrieve a single disk block (32 KByte size) from a random location of a disk that rotates at 10,000 RPM, have a data transfer rate of 10 MBytes/s and have an 8 ms average seek time? what is the bandwidth (bytes/sec) that can be achieved, if the application requests are randomly distributed
25. With the current head position of disk as shown in the figure and for an I/O request stream of reading blocks 21, 17, 11, 7, 13, 27 what will be the order for the following scheduling policy



- i. SSTF
- ii. SPTF ((assume seek is much faster than rotation))

26. Consider a file system with 4096 byte blocks, 32-bit disk interface and file block pointers. Each file has 13 direct pointers, 4 singly-indirect pointers, a doubly-indirect pointer, and a triply-indirect pointer. In the following be explicit about your work:

- What is the maximum disk size that can be supported?.
- What is the maximum file size?
- Why use a combination of direct, double and triple indirection to locate all file blocks?  
Since majority files are small which need only few direct pointers but at the same time making the option available to accommodate very big files.
- Give some reasonable assumptions and compute the number of inodes that can fit into a disk block.

27. Assume there are no I-nodes or data blocks currently in memory and that all directory data is held in the first direct block of I-nodes representing the directories in question. Show various (and their order) disk accesses that are required to create the file ``/fred/mary/joe".

| Data<br>bitmap | Inode<br>bitmap | Root<br>inode | Fred<br>inode | Marry<br>inode | Joe<br>inode | Root<br>data | Fred<br>data | Marry<br>data |
|----------------|-----------------|---------------|---------------|----------------|--------------|--------------|--------------|---------------|
|                |                 | read          |               |                |              |              |              |               |
|                |                 |               |               |                |              | read         |              |               |
|                |                 |               | read          |                |              |              |              |               |
|                |                 |               |               |                |              |              | read         |               |
|                |                 |               |               | read           |              |              |              |               |
|                |                 |               |               |                |              |              |              | Read          |
|                | read            |               |               |                |              |              |              |               |
|                | Write           |               |               |                |              |              |              |               |
|                |                 |               |               |                |              |              |              | Write         |
|                |                 |               |               |                | read         |              |              |               |
|                |                 |               |               |                | write        |              |              |               |
|                |                 |               |               | write          |              |              |              |               |

